

Serial Communications

Chapter Contents

Overview	238
Concepts	238
Why?	238
How?	239
Serialization	239
Symbol Timing	240
Message Framing	240
Error Detection	241
Acknowledgments	241
Media Access Control	241
Addressing	242
Development Tools	242
Software Structures for Communication	244
Supporting Asynchronous Communication	244
Queue Implementation	246
Queue Use	249
Serial Communication Protocols and Peripherals	249
Synchronous Serial Communication	249
Protocol Concepts	249
STM32F091RC SPI Peripherals	251
Example: SPI Loopback Test	255
Asynchronous Serial Communication	257
Protocol Concepts	257
STM32F091RC USART Peripherals	258
Example: Communicating with a PC	262
Inter-Integrated Circuit Bus (I ² C)	269
Protocol Concepts	269
STM32F091RC I2C Peripherals	273
Example: Polled I ² C Communications with an Inertial Sensor	274
Summary	281
Exercises	281
References	282

Overview

Serial communication simplifies the creation of complex embedded systems from separate hardware components. In this chapter we examine the basic ideas of wired serial communication and three common types of protocols: synchronous serial, asynchronous serial, and Inter-Integrated Circuit Bus (I²C). For each protocol we examine the peripherals and supporting code. We discuss methods to structure the software to handle the lack of timing synchronization between program and communication activity. We also examine tools that simplify the development of systems using such protocols.

serial

Organization in which parts of an item are sequentially available or active, but not simultaneously

Concepts

Why?

Embedded systems are made of multiple hardware components that must communicate with each other. Why communicate **serially**?

Some of these components are integrated into the MCU. Because they are on a single chip, these components can communicate directly over the system's data bus using their native data type (e.g. bytes, 16-bit half-words, or 32-bit words). These internal buses are called **parallel** because they have a separate wire or signal for each bit of the data type. As a result they can send a data item in a single transaction, resulting in fast transfers.

parallel

Organization in which all parts of item are simultaneously available or active

Off-chip components are needed for many embedded systems. Using parallel communication to reach these off-chip parts may be fast but has disadvantages that grow as the communication distance or speed increase. First, the packages for the MCU and off-chip components must have a pin (or pad) for each bus signal. For example, a 32-bit bus requires 32 pins for data, multiple pins for addressing, and control lines to signal read or write operations. These large pin counts increase the package size and cost. Second, the printed circuit board (PCB) becomes more complex. A parallel bus has many signals that must be routed in a limited area. High-speed buses require more careful design to ensure signal timing integrity. Third, if the bus must be situated off the PCB, then the connectors and cables must be large enough to provide one connection or wire per signal.

One way to address these challenges is to transmit the data serially rather than in parallel; we send a **symbol** representing one or several bits (rather than all the bits) at a time. For example, sending a 32-bit word eight bits at a time would take four transmissions. Sending an 8-bit byte one bit at a time would take eight transmissions.

symbol*Organization in which all parts of time are simultaneously available or active*

Serialization reduces the number of pins needed on a chip package, the number of contacts in a connector, and the number of wires in cables. This enables smaller chip packages and connectors, which are less expensive and often significantly lighter. The circuit board design is simplified because there are fewer signals to route. Because of these benefits, most MCUs support serial communication, and there are many compatible components available.

How?

There are many different decisions to make when deciding how to communicate serially between computing systems. The Open System Interconnection (OSI) model from the International Organization for Standardization (ISO) defines seven layers of a communication system [1]. The three lowest layers are relevant for this chapter:¹

- The **physical** layer (layer 1) specifies how symbols are represented on the communication medium (e.g. wire) as voltages or currents.
- The **data link** layer (layer 2) has two parts. The **media access control** determines when a node can transmit, defining how time on the bus is shared. The **logical link control** determines how the receiver identifies the start and end of a message from a stream of symbols on the physical layer. It also defines how errors are detected.
- The **network** layer (layer 3) defines how to address nodes, split up long data to fit into multiple messages, and handle errors, and other characteristics.

With this context, we can now examine the fundamental concepts of serial communication.

Serialization

Serialization is the conversion of data from a parallel to a serial format, whereas **deserialization** is the reverse. Each MCU and serial peripheral has internal shift registers to perform this. The serialized information is a stream of symbols that represent the data and control information. The communication protocols examined here can store one bit of data per symbol, but other examples improve speed by encoding multiple data or control bits in a single symbol. The rate at which the symbols are transmitted is called the **baud rate**.

A transmitting device uses a parallel-in-serial-out shift register that is first loaded with data from a multibit parallel input bus. To serialize the parallel data, a clock circuit applies a series of pulses to shift the data one bit position at a time and stream out the serial output.

¹ The upper layers of the OSI model (**transport, session, presentation, application**) deal with higher level issues such as reliable communication and security.

A receiving device uses a serial-in-parallel-out shift register. To deserialize the serial data, a clock circuit applies a series of pulses to load the shift register one bit at a time from the serial input. After all bits of the shift register are loaded, the data can be read from the parallel output bus.

serialization

The process of converting information from parallel to serial form

deserialization

Conversion of information from serial to parallel form

baud rate

Rate at which communication symbols are transmitted. Also called symbol rate.

Symbol Timing

In order for the deserialization to work reliably, the receiver's clock pulses need to be applied at the right times. The serial data line must be sampled once per symbol (at the baud rate). This sampling should be at the middle of the symbol time in order to avoid signal transitions, where the signal may be corrupted due to noise or slow circuits.

With a **synchronous** approach the transmitter's clock signal is connected directly to the receiver's clock input. This approach is reliable but requires three signal connections: clock, data, and ground.

With an **asynchronous** approach, the transmitter provides no clock signal. Instead, the receiver has a clock running at the same frequency as the transmitter that determines **when** to sample the signal line to capture each symbol.

synchronous

Activities which are synchronized with each other, or a protocol which sends clocking information

asynchronous

Activities that are not synchronized with each other, or a protocol that does not send clocking information

Message Framing

How does the asynchronous receiver synchronize its clock with the transmitter's clock if there is no connection? This is done by adding a **framing symbol** to indicate the start of the message. The receiver clock starts running when it detects the framing symbol (such as a **start bit**) and then samples the input in the middle of each following symbol time. A framing symbol may also be used to indicate the end of a message (e.g. a **stop bit** or a **stop symbol**).

framing symbol*Symbol used to indicate start or end of message*

Error Detection

Communication links are vulnerable to noise if they are long or poorly shielded. Some communication protocols add information to each message to detect transmission errors. Common error detection methods are **parity bits**, **checksums**, and **cyclic redundancy checks** [2]. With some protocols, there is a dedicated receive error notification bit in each message, allowing a receiver to notify the transmitter (and all other receivers) that the message was received incorrectly.

Acknowledgments

Some protocols include an **acknowledgment** field within each message, allowing a receiver to signal successful message reception. Other protocols may use a separate acknowledgment message, or nothing at all.

acknowledgment*Device response indicating successful reception of message*

Media Access Control

If there are multiple possible transmitters on a single communication bus, then the transmitters need to follow a set of rules to share the time on the bus. Without these rules, two transmitters might try to send at the same time, creating a collision which garbles both messages. These rules are called a **media access control (MAC)** method and determine when a device can transmit, and in turn help determine how much of the bus bandwidth can be used for actual data. One common approach uses a coordinating node which transmits as needed, and other nodes which follow a strict “speak when you are spoken to” rule.

This approach is commonly called Master/Slave, and is used in many popular communication protocols, including two covered in this chapter. SPI and I²C were created in the 1980’s and use the terms master and slave in names of devices, signals and control and status registers. Thus there is a very large installed base of devices, software, documentation and tools which use these original names. If it had been created more recently, one hopes that it would have used terms which are more culturally sensitive, such as controller and peripheral. In fact, there are calls to rename the SPI device and signal names in this way. For example, MISO (Master In, Slave Out) becomes CIPO (Controller In, Peripheral Out), MOSI becomes COPI, and SS becomes CS (chip select).

This textbook must strike a balance between two competing concerns. References to slaves will make many people feel uncomfortable and should be avoided when possible. On the other hand, simpler is better when learning. Alternatives and translations complicate an already

difficult matter. Given these factors, this textbook uses the industry-standard terms but asks the reader to consider their unintended consequences in order to avoid making mistakes of this type in the future.

media access control (MAC)

Rules controlling when a node can transmit a message on shared media

Addressing

If there are multiple possible receivers on a single communication bus, then the transmitter may need to specify which receiver is the target of the communication. This information can be sent on separate select signals, or it may be included in the message itself.

Development Tools

There are several tools that will make it easier to develop embedded systems that use serial communication protocols. The most basic tool is an **oscilloscope**, which shows a signal's voltage over time. The developer needs to interpret the signals to determine the communication activity. The encoded data and precise timing relationships of serial communication can make it difficult to debug embedded systems. For example, what do the signals in Figure 8.1 mean?



Figure 8.1 Oscilloscope shows voltage levels of signals over time.

Manual interpretation is often slow, tedious, and error-prone, so a much better tool is a **logic analyzer** with a **protocol decoder**. This tool interprets the signals automatically and displays them in an easily understood readable format.

Figure 8.2 shows the previous waveforms and their meaning when decoded according to the I²C communication protocol. First there is a write to device 1D of the value 0x 01. Second there is a read from device 1D of 6 data bytes (0x 00, 0x 90, 0x FF, 0x F8, 0x 41, and 0x 70). Note that this program shows hexadecimal values with a prefix of **h**, whereas the C language uses a prefix of **0x**. The values are the same regardless of the prefix.

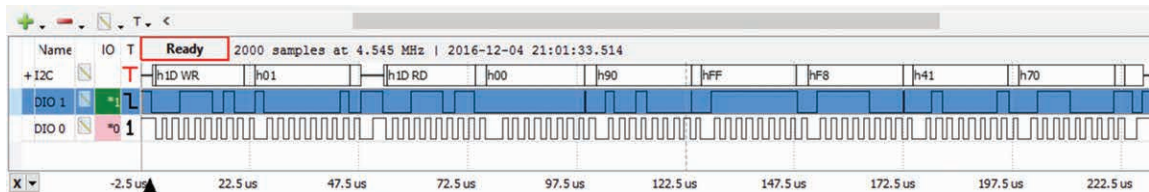


Figure 8.2 Logic analyzer interprets bus signals and displays message information.

A logic analyzer may also save the received data for further analysis or processing. The developer typically needs to configure the protocol decoder to match the protocol in use, for example, setting the data rate or identifying a clock signal. Some oscilloscopes may also include logic analyzers and protocol decoders.

There are PC-based oscilloscopes and logic analyzers available, which are often less expensive than stand-alone devices. The Analog Discovery 2 from Digilent, Inc. is shown in Figure 8.3 and can serve as a logic analyzer, oscilloscope, waveform generator, and many other types of test equipment [3]. It offers a 16-input logic analyzer with support for various serial communication protocols. The PC-based software provides the graphical interface seen in Figure 8.1 and Figure 8.2.

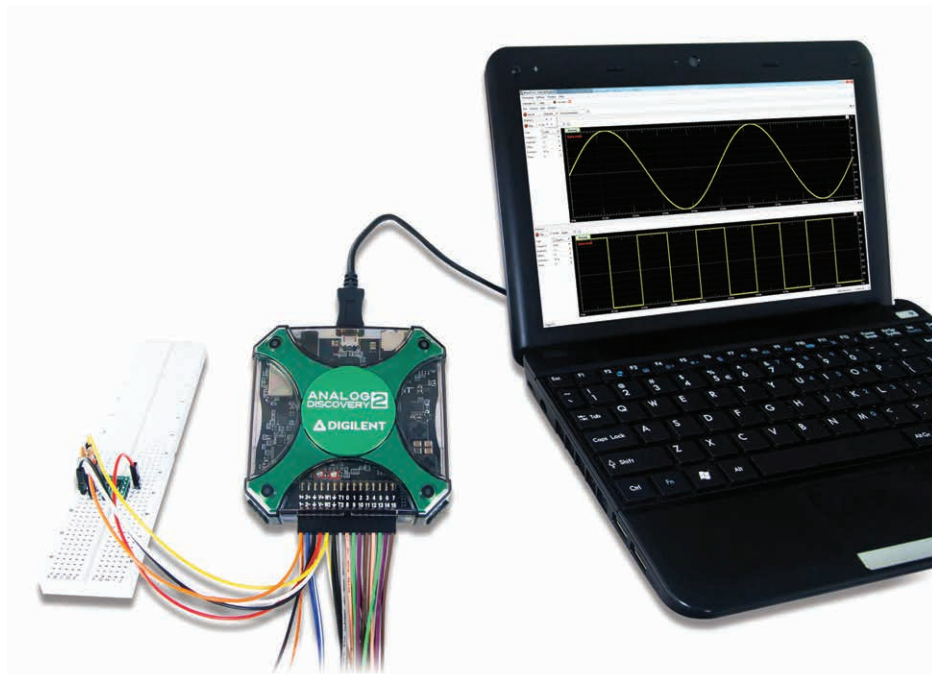


Figure 8.3 Analog Discovery 2 device serves as 2-channel oscilloscope, 16-channel logic analyzer and other tools. User interface program runs on a PC with USB connection. Image courtesy Digilent, Inc.

Another useful tool is a PC-based bus interface, which allows a developer's PC to send and receive messages on the bus. One example is the Bus Pirate, which connects to the PC with a virtual serial port over USB [4]. The developer uses a terminal emulator program on the PC to control the Bus Pirate through a serial console interface. The Bus Pirate has many features: support for various serial communication protocols, scripting, and the ability to program flash memory and MCUs. It also offers switchable power supplies, ADC input, and PWM output.

Software Structures for Communication

Communications make it harder to share the CPU for two reasons. First, we don't know **which** program instruction will be executing when data is received. The program and the data reception are asynchronous. This means there is no timing relationship between the program's progress and data reception. Second, it takes a significant amount of time to send one data item. In this time, the program can execute many instructions, so it is not clear at **which** program instruction the transmitter will be ready to send another data item.² So we consider transmission to be asynchronous as well.

Supporting Asynchronous Communication

We would like to create a timing relationship between the program and communication events. One approach is to use polling. For example, the program could spin in a loop until data has been received. Although simple, polling makes the CPU harder to share, as discussed in Chapter 3.

Another approach is to use interrupts, providing event-triggered processing. Every time the peripheral receives a data item, the peripheral will signal an interrupt and the CPU will run an ISR to get it. Every time the peripheral is ready to send an item, it will signal an interrupt and the CPU will run an ISR to start transmitting the next data item. The ISR may also execute a **callback** function (e.g. when all requested data items have been received).

We will split the work between the ISRs and the program's tasks to provide good responsiveness and to simplify scaling up the program later as we add other features. Recall that the longer an ISR takes to run, the longer all other ISRs can be delayed. To reduce these delays, the ISR will perform the most time-critical operations with the data and leave other processing for lower-priority code (e.g. task code). To do this we must somehow store data between the ISRs and the rest of the program.

This stored data will flow from the producer, which generates the data, to the consumer, which uses the data. For transmission, the producer is the task that creates the data to send, whereas the consumer is the transmit ISR that loads the peripheral's transmit data buffer. For reception, the producer is the receive ISR that reads the peripheral's received data buffer, whereas the consumer is the task that uses that data.

We will use a data buffer to hold the producer's output data until it is read by the consumer. In most cases we want the data to be delivered to the consumer in the order it was produced, using a first-in, first-out (FIFO) ordering. A **queue** is another name for a buffer with FIFO ordering. To **enqueue** an item is to add it to the queue. To **dequeue** an item is to remove it from the queue.

² Technically they are synchronous, but in a system with even minor complexity identifying the specific instruction(s) requires extensive (and impractical) program timing analysis each time the program is built. And there will be multiple instructions if there are multiple control flow paths.

As communication systems typically provide both transmission and reception, we will need a queue for each direction. Figure 8.4 shows typical hardware and software components used for queued, interrupt-driven communication. At the left, the serial communication peripheral interfaces with the communication bus signals. It generates an interrupt after receiving a data item or when it is ready to transmit another.

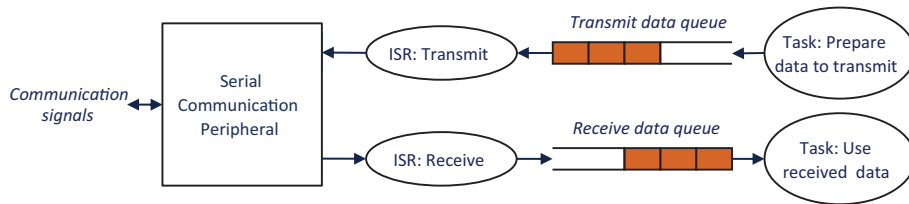


Figure 8.4 Software structure for interrupt-driven queued communication.

Figure 8.5 shows the case where the peripheral is ready to transmit. The transmit ISR will dequeue the next data item from the transmit data queue and place it in the peripheral for transmission. A task enqueues data the transmit data.

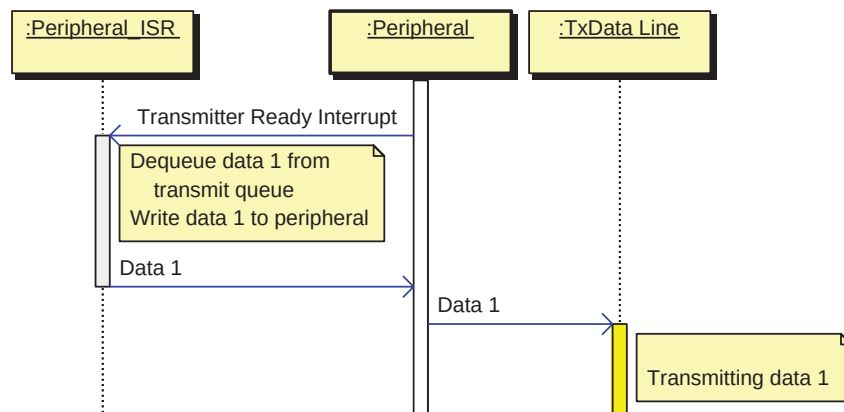


Figure 8.5 Sequence of activities for data transmission.

Figure 8.6 shows the case where the peripheral has received data. The receiver ISR will read the data from the peripheral and enqueue it into the received data queue. A task will later dequeue the received data and use it.

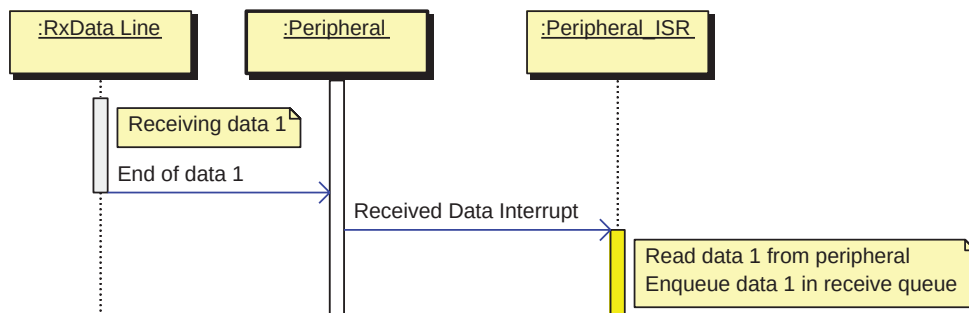


Figure 8.6 Sequence of activities for data reception.

Queue Implementation

We will create a queue data structure to hold data before the receiving code can process it. There are various ways to implement a queue, but the approach we use here is efficient and simple.

The data is stored in an array, as shown in Figure 8.7. Rather than move all of the data each time an item is added or removed, we will use indexes to keep track of the head and tail. The head indicates the oldest data element, which will be read in the next dequeue operation. The tail indicates the free space to use when enqueueing the next element.

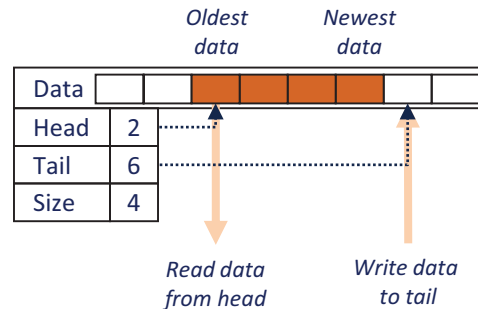


Figure 8.7 Queue data structure contains array to hold data, indexes for head and tail, and used size information.

The data structure type definition for the queue is shown in Listing 8.1. When defining a queue variable of this type, be sure to define it as volatile because it will be shared between ISRs and regular code. This will prevent the compiler from performing risky optimizations on this type of data structure. Instead, the compiler will generate code that forces the CPU to load the data values from memory each time the source code references the data.

```
#define Q_MAX_SIZE (256)
typedef struct {
    uint8_t Data[Q_MAX_SIZE];
    // Index of oldest data element
    unsigned int Head;
    // Index of next free space
    unsigned int Tail;
    // Number of elements in use
    unsigned int Size;
} Q_T;
```

Listing 8.1 Data structure definition for queue.

How many elements should the queue be able to hold? In this example, up to 256 elements can be held, as defined by `Q_MAX_SIZE`. The correct value depends on how often an item may be enqueued, how long a delay can occur before the first item is dequeued, and how often subsequent items can be dequeued. We use a size field variable to track the number of elements currently in use. Note that we could instead calculate this value from the difference between the head and tail indices.

The enqueue and dequeue operations are shown in Listing 8.2. Adding an element to the queue will update the tail, whereas removing an element will update the head. The update operations are simply increments that are wrapped around to zero when the end of the array is reached. This type of implementation is called a ring buffer. Making the number of array elements a power of two will simplify the wrapping operation to simply masking off bits (e.g. with a bitwise and operation). Otherwise the modulus (remainder) operation will need to be performed, which is likely to be computationally expensive.

```
int Q_Enqueue(Q_T *q, uint8_t d) {
    uint32_t masking_state;
    // If queue is full, don't overwrite data, but do return an error code
    if (!Q_Full(q)) {
        q->Data[q->Tail] = d;
        // Protect operations from preemption
        // Save current masking state
        masking_state = __get_PRIMASK();
        // Disable interrupts
        __disable_irq();
        // Update variables
        q->Tail = (q->Tail+1) % Q_MAX_SIZE;
        q->Size++;
        // Restore interrupt masking state
        __set_PRIMASK(masking_state);
        return 1; // Success
    } else
        return 0; // Failure
}

uint8_t Q_Dequeue(Q_T *q) {
    uint32_t masking_state;
    uint8_t t = 0;
    // Check to see if queue is empty before dequeuing
    if (!Q_Empty(q)) {
        t = q->Data[q->Head];
        q->Data[q->Head] = '_'; // Empty unused entries for debugging
        // Protect operations from preemption
        // Save current masking state
        masking_state = __get_PRIMASK();
        // Disable interrupts
        __disable_irq();
        // Update variables
        q->Head = (q->Head+1) % Q_MAX_SIZE;
        q->Size--;
        // Restore interrupts
        __set_PRIMASK(masking_state);
    }
    return t;
}
```

Listing 8.2 Functions for enqueueing and dequeuing data.

This queue is expected to share data between multiple threads with preemption due to interrupts. This introduces the risk of data corruption due to non-atomic access to the queue fields. As discussed in Chapter 3, for an Arm Cortex-M (or any other load/store architecture) processor to modify a variable stored in memory, that variable must be loaded from memory into a register first. The register can then be modified and stored back to memory. This sequence of code is a critical section that is vulnerable to corruption based on timing. Consider the case where a function has called `Q_Dequeue`. The `Size` field has a value of N , and is loaded from memory into a register to be decremented to $N-1$. Before the new value is stored to memory, another character arrives, causing the CPU to preempt `Q_Dequeue` and execute the ISR. The ISR calls `Q_Enqueue`, which loads N (the old value of `Size`) from memory into a register, increments it to $N+1$, and stores it to memory. `Q_Enqueue` completes, the ISR completes, and `Q_Dequeue` resumes executing, storing the value $N-1$ to memory. `Size` is now wrong: it should be N , but is $N-1$.

Another type of critical section results when multiple variables must be updated without interruption to be correct. For example, if we didn't have a `Size` field, each time we wanted to determine the size of the queue, we would need to access both `Head` and `Tail` to calculate $(\text{Tail} - \text{Head}) \% \text{Q_MAX_SIZE}$. All accesses to `Head` or `Tail` would create critical sections.

We protect the critical section of the code by disabling interrupts during its execution. To do this, our code first saves the current interrupt masking state using `__get_PRIMASK`, and then disables interrupts using `__disable_irq`. After executing the critical section, the code restores the previous interrupt masking state using `__set_PRIMASK`. These functions are defined in CMSIS-CORE.

Further examination would show that none of the other fields in the `Q_T` structure is vulnerable to `Q_Enqueue` preempting `Q_Dequeue`, or `Q_Dequeue` preempting `Q_Enqueue`. However, if we allow `Q_Enqueue` to preempt `Q_Enqueue`, the operations on `Tail` and `Data` become critical sections and must be protected. Similarly, allowing `Q_Dequeue` to preempt `Q_Dequeue` makes the operations on `Head` and `Data` critical sections that must be protected.

Finally, the code for initialization and status checks is shown in Listing 8.3.

```
void Q_Init(Q_T *q) {
    unsigned int i;
    for (i = 0; i < Q_MAX_SIZE; i++)
        // To simplify our lives when debugging
        q->Data[i] = 0;
    q->Head = 0;
    q->Tail = 0;
    q->Size = 0;
}

int Q_Empty(Q_T *q) {
    return q->Size == 0;
}

int Q_Full(Q_T *q) {
    return q->Size == Q_MAX_SIZE;
}

int Q_Size(Q_T *q) {
    return q->Size;
}
```

Listing 8.3 Functions for queue initialization and status checks.

Queue Use

In order to use the queue, we need to decide whether we want our code to block and wait for data to be available (for dequeuing) or space to be available (for enqueueing). If our code should not block, then we will simply test the condition to determine whether to perform the queue operation. If the condition is true, then the code continues with the queue operation. If the condition is not true, then the code needs to defer this work for later. The actual approach will depend on the specifics of the application.

If our code should block, then we use a loop to wait for the appropriate condition to become true before performing the queue operation. Note that this blocking operation does not share the processor. When using a finite state machine to allow other code to run, this test code should be a separate state. If the condition is not true yet, the state should end and be repeated on the next call. If the condition is true, the code can continue with the queue operation and then advance to the next state. When using a task scheduler, if the condition is not true, the code should yield the processor briefly to other tasks.

Operations that might block should not be performed in interrupt handlers because they can introduce sporadic timing delays that are difficult to repeat and therefore debug. Instead, the handler should be prepared to discard the data (or handle it in some other way) and signal an error to the rest of the application.

Serial Communication Protocols and Peripherals

We can now examine three types of serial communication protocols and the corresponding peripherals. We start with a basic approach (synchronous serial), then advance to asynchronous serial and finish with I²C, which offers addressing and other higher-level features. Figure 8.8 shows the connections on the Nucleo-64 board for the SPI, USART and I²C examples in this chapter.

Synchronous Serial Communication

Protocol Concepts

Serial peripheral interface (SPI) is a type of synchronous serial communication with a master and one or more slaves (Figure 8.9). Typically the MCU is the master and peripheral devices are slaves. Some common slave SPI devices are ADCs, accelerometers, LCD controllers, and magnetometers. An MCU might instead be configured to operate as a slave, for example, to create a smart MCU-based peripheral subsystem in a larger system with a different MCU serving as the master.

SPI communication between a master and slaves uses three signals (clock and two data signals), a select signal for each slave and ground.

- The clock signal (SPSCK or SCK) indicates when data is to be sampled.
- The MOSI data signal is the master output and slave input.
- The MISO data signal is the master input and slave output.
- The master asserts the select line of the slave targeted for communication. This signal is typically active-low.

```
void Test_SPI_Loopback(void) {
    uint8_t out = 'A';
    uint8_t in;
    while (1) {
        in = SPI_Send_Receive_Byte(out);
        if (in != out) // Red: error, data does not match
            Control_RGB_LEDS(1, 0, 0);
        else // Green: data matches
            Control_RGB_LEDS(0, 1, 0);
        out++;
        if (out > 'z')
            out = 'A';
    }
}
```

Listing 8.6 Code to test SPI transmission and reception

The `main` function in Listing 8.7 initializes SPI1 and the GPIO ports for the LEDs, after which it calls the loopback test code.

```
int main(void) {
    Set_Clocks_To_48MHz();
    Init_GPIO_RGB();
    Init_SPI1();
    Test_SPI_Loopback();
}
```

Listing 8.7 Code to initialize MCU and run SPI loopback test.

Asynchronous Serial Communication

Protocol Concepts

Asynchronous serial communication works without a shared clock signal. Instead, both the transmitter and receiver have clock generators that must be configured to run at the same speed. The generic name for a peripheral that supports this is *universal asynchronous receiver/transmitter (UART)*. Such a peripheral often includes support for both asynchronous (no shared clock) and synchronous (with a shared clock) communication, so it may be called a *universal synchronous/asynchronous receiver/transmitter (USART)*.

universal asynchronous receiver/transmitter (UART)

Peripheral for asynchronous serial communications

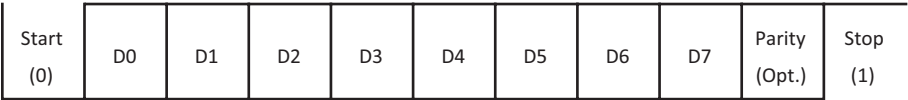


Figure 8.15 Contents of asynchronous serial communication message. The parity bit is optional, and there may be more than one stop bit.

Asynchronous communication typically allows transmission and reception to occur independently. This is different from SPI, where both the master and slave transmit and receive simultaneously. A USART contains separate hardware for the transmitter and receiver.

A general USART message is shown in Figure 8.15 and contains these fields:

- The message begins with a start bit (always a logic zero) which indicates the beginning of the message. Because there is no clock signal transmitted with the data, the start bit lets the receiver's clock synchronize to the incoming message and start message reception correctly.
- The data field of the message is typically 8 bits, but other sizes may also be supported. Data may be sent LSB first or MSB first.
- Parity helps detect errors in data transmission and can be enabled on UARTs. The parity of a message is determined by the total number N of one bits in the data character and the parity bit. If N is even, then the message has even parity. If N is odd, then the message has odd parity. The UARTs of the transmitter and receiver are configured to expect each received message to have the correct parity (e.g. odd). When parity is enabled, the transmitter computes the parity of the data and then adjusts the parity bit to one or zero to match the specified communication parity (e.g. odd). The receiver calculates the parity of the received data and parity bit, and verifies it matches the expected parity (e.g. odd). If it does not (e.g. there were an even number of ones received in the message), then the UART signals a parity error for the software to handle.
- One or more stop bits with a value of logic one are added to help the receiver detect timing errors. If a zero is received for any stop bit, then the receiver will indicate an error so the software reacts appropriately.

The USART hardware is more complex than the SPI hardware, with the addition of the hardware for the receiver clock generator, framing, and parity. Figure 8.16 shows a sequence of three bytes ("1_2") transmitted by a UART.

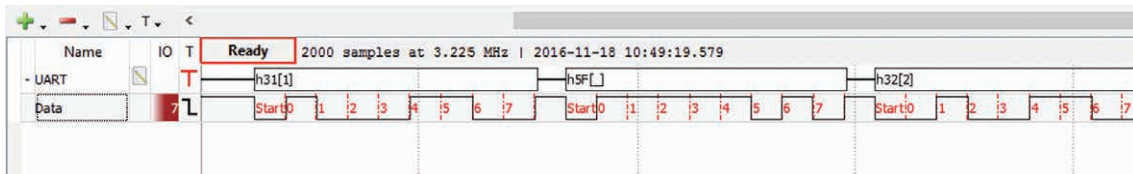


Figure 8.16 Example of serial data ("1_2") captured by logic analyzer.

STM32F091RC USART Peripherals

The STM32F091RC microcontroller has eight USART peripherals, with USART1 offering additional features. Clock gating must be enabled for the USART peripheral using the RCC register `RCC_APB1ENR` (for USART 2, 3, 4 and 5) or `RCC_APB2ENR` (for USART 1, 6, 7 and 8).

The USART contains a baud rate generator, a transmitter, and a receiver. The baud rate generator divides down an input clock signal to a lower frequency for USART communication. The transmitter uses a shift register to convert the input data (from `USARTx_TDR`), framing bits and optional parity information into a serial stream of bits. The receiver uses a similar shift register to convert the serial bit stream into a set of parallel bits. An edge detection circuit is used to identify the start bit and start shifting data in at the correct time. Error detection circuitry identifies framing, parity, and other errors.

The following fields in USART1_CR1 (shown in Figure 8.17) are frequently used:

- The M0 and M1 fields determine whether data is seven bits long (M1 = 1 and M0 = 0) or eight bits long (M1 = 0 and M0 = 0) or nine bits long (M1 = 0 and M0 = 1).
- When the PCE field is one, it enables parity generation and checking.
- When parity is enabled (PCE = 1), the PS field selects even parity (0) or odd parity (1). An interrupt can be generated in case of error by setting PEIE.
- When set to one, TE, RE and UE enable the transmitter, receiver, and USART device respectively.
- When set to one, RXNEIE, TCIE, and TXEIE control whether certain events trigger interrupts, and are discussed in subsequent pages.

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Field	Res.	Res.	Res.	M1	EOBIE	RTOIE	DEAT					DEDT				
Reset				0	0	0	0	0	0	0	0	0	0	0	0	0
Access				rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Field	OVER8	CMIE	MME	M0	WAKE	PCE	PS	PEIE	TXEIE	TCIE	RXNEIE	IDLEIE	TE	RE	UESM	UE
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Access	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Figure 8.17 Contents of USART1_CR1 control register.

The following fields in USART1_CR2 (Figure 8.18) are frequently used:

- The STOP field selects the length of the number of stop bit (0.5, 1, 1.5 or 2 stop bits).
- The MSBFIRST field determines if the data are sent with the MSB first (1) or with the LSB first (0).
- When set to one, RXINV inverts received data and TXINV inverts transmitted data.

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Field	ADD								RTOEN	ABRMOD	ABREN	MSBFIRST	DATAINV	TXINV	RXINV	
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Access	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Field	SWAP	LINEN	STOP		CLKEN	CPOL	CPHA	LBCL	Res.	LBDIE	LBDL	ADD M7	Res.	Res.	Res.	Res.
Reset	0	0	0	0	0	0	0	0		0	0	0				
Access	rw	rw	rw	rw	rw	rw	rw	rw		rw	rw	rw				

Figure 8.18 Contents of USART1_CR2 control register.

The EIE field in USART1_CR3 (Figure 8.19) controls whether errors will trigger interrupts.

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Field	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	WUFIE	WUS			SCARCNT		
Reset										0	0	0	0	0	0	
Access										rw	rw	rw	rw	rw	rw	

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Field	DEP	DEM	DDRE	OVRDIS	ONEBIT	CTSIE	CTSE	RTSE	DMAT	DMAR	SCEN	NACK	HDSEL	IRLP	IREN	EIE
Reset	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
Access	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw*	rw*	rw	rw	rw	rw

Figure 8.19 Contents of USART1_CR3 control register.

The USARTx_TDR register is used to access the transmit buffer and the USARTx_RDR register is used to access the receive buffer. A write to USARTx_TDR writes to the transmit buffer whereas a read from USARTx_RDR reads from the receive buffer.

Baud Rate Generator

Each USART has a clock that determines the baud rate for the transmitter and the receiver. The 16 bits of the USARTx_BRR register are used to divide the bus clock down to the desired baud rate.

To reduce vulnerability to noise, the receiver samples the RX signal multiple times per bit. The bit's value is determined to be the most common bit value in those samples. This is called oversampling. After MCU reset, the oversampling factor is 16 because the OVER8 field of USARTx_CR1 is zero. Setting OVER8 to one would reduce the oversampling factor to 8.

With 16x oversampling, the bus clock (e.g. 48 MHz) is divided by the 16-bit value from USARTx_BRR to determine the baud rate:

$$\text{Baud Rate} = \frac{f_{\text{BusClock}}}{\text{USARTx_BRR}}$$

To get a specific baud rate with 16x oversampling, the USARTx_BRR value is:

$$\text{USARTx_BRR} = \frac{f_{\text{BusClock}}}{\text{Baud Rate}}$$

When the oversampling factor is 8, a slightly different approach is used; please refer to the reference manual for details.

Status and Interrupts

The USART peripheral indicates when events have occurred using status flags and interrupts. The Interrupt and Status Register (USARTx_ISR) shown in Figure 8.20 holds the following flags for transmission:

- The TXE flag indicates when the transmit data buffer has room to accept another character (via USART_TDR).
- The TC flag indicates when transmission is complete.

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Field	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	REACK	TEACK	WUF	RWU	SBKF	CMF	BUSY
Reset										0	0	0	0	0	0	0
Access										r	r	r	r	r	r	r

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Field	ABRF	ABRE	Res.	EOBF	RTOF	CTS	CTSIF	LBDF	TXE	TC	RXNE	IDLE	ORE	NF	FE	PE
Reset	0	0		0	0	0	0	0	1	1	0	0	0	0	0	0
Access	r	r		r	r	r	r	r	r	r	r	r	r	r	r	r

Figure 8.20 Contents of USARTx_ISR interrupt and status register.

USARTx_ISR holds the following flags for reception:

- The RXNE flag indicates there is data to read from the receive buffer (via USARTx_RDR).
- The IDLE flag indicates the receive data line is idle.

USARTx_ISR holds the following flags for errors:

- The PE flag indicates a parity error was detected during reception.
- The FE flag indicates a framing error was detected during reception.
- The ORE flag indicates that the receive buffer's data was not read before new data was received (overrun error).
- The NF flag indicates noise was detected by the receiver. This can occur when oversampling is enabled, in which the receiver samples each bit multiple times.

These flags are reset by writing 1 to the corresponding bit in the Interrupt flag Clear Register USARTx_ICR (Figure 8.21).

Bit	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Field	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	WUCF	Res.	Res.	CMCF	Res.
Reset												0			0	
Access												rc_w1			rc_w1	

Bit	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Field	Res.	Res.	Res.	EOBCF	RTOCF	Res.	CTSCF	LBDCF	Res.	TCCF	Res.	IDLECF	ORECF	NCF	FECF	PECF
Reset				0	0		0	0		0		0	0	0	0	0
Access				rc_w1	rc_w1		rc_w1	rc_w1		rc_w1		rc_w1	rc_w1	rc_w1	rc_w1	rc_w1

Figure 8.21 Contents of USART1_ICR interrupt flag clear register.

A USART can generate an interrupt in response to these three types of events if the appropriate interrupt enable bits are set.

- Transmit events: Setting TXEIE in USARTx_CR1 to one enables interrupts when the transmit data register is empty (TXE is one). TCIE enables interrupts when transmission is complete (TC is one).
- Receive events: Setting RXNEIE to one enables interrupts when the receive data register is full (RXNE is one). Other receive events are possible as well, but are not discussed further here.
- Error events: If EIE field in USARTx_CR3 is set, an interrupt is generated if one of the following flags is set: NF, ORE or FE. An interrupt is generated in case of parity error (PE is set) if the PEIE field in USARTx_CR1 is set.

Other Features

There are many other features available that we do not cover here, such as single-wire mode, loop mode, transmit data inversion, communication protocol support, wake-up on idle line, address mark or match address. Further information is available in the reference manual [5].

Example: Communicating with a PC

Let's see how to use asynchronous serial communication between the STM32F091RC MCU and a terminal program on a PC. We'll use the debug MCU as a bridge. It provides a virtual serial port service to the PC over its USB connection, and asynchronous serial communications to the target MCU. The ST-Link MCU's USART is configured to run at 9600 baud, with 8-bit data, no parity and one stop bit.

The Nucleo-F091RC development board connects the target MCU's USART2 through PA2 and PA3 with a USART in the ST-Link debug MCU, as shown in Figure 8.22. Note that by default these signals do not go to the PA2 and PA3 pins on connectors CN9 and CN10.⁴ To monitor the TX and RX signals, connect your logic analyzer or oscilloscope to connector CN3 on the ST-Link portion of the Nucleo board as shown in Figure 8.8.

⁴ They can be connected by desoldering bridges SB13 and 14 while soldering SB62 and 63 as described in the Nucleo User Manual [10].

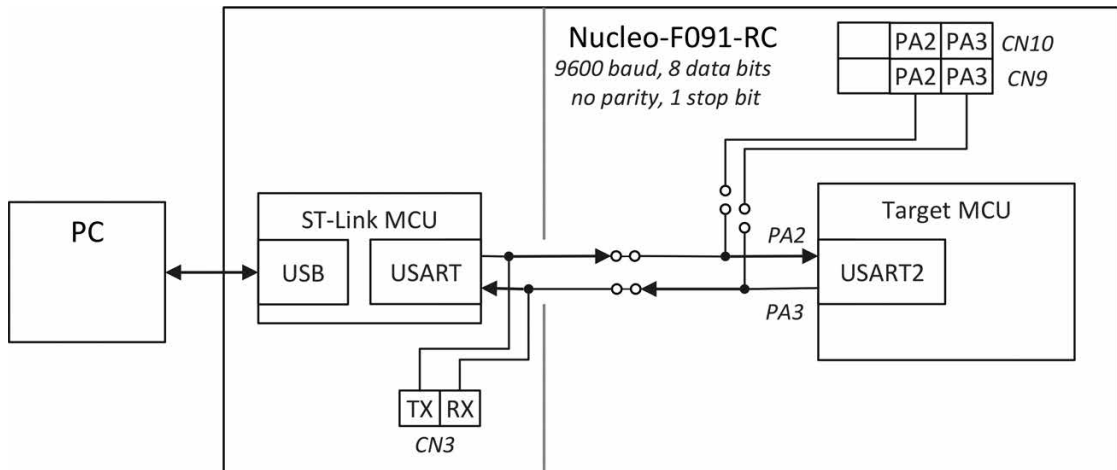


Figure 8.22 The ST-Link MCU provides a virtual serial port between PC and target MCU's USART2. To monitor the TX and RX signals with a logic analyzer, connect to TX and RX on CN3 instead of the PA2 and PA3 on CN9 or CN10.

```
#define F_USART_CLOCK (48UL*1000UL*1000UL) // 48 MHz

void Init_USART2(void) {
    RCC->APB1ENR |= RCC_APB1ENR_USART2EN;
    RCC->AHBENR |= RCC_AHBENR_GPIOAEN;

    // GPIO A pin 2 and 3 in alternate function 1 (USART2)
    // Set mode field to 2 for alternate function
    MODIFY_FIELD(GPIOA->MODER, GPIO_MODER_MODER2, ESF_GPIO_MODER_ALT_FUNC);
    MODIFY_FIELD(GPIOA->MODER, GPIO_MODER_MODER3, ESF_GPIO_MODER_ALT_FUNC);
    // Select USART2 (AF = 1) as alternate function
    MODIFY_FIELD(GPIOA->AFR[0], GPIO_AFR_L_AFSEL2, 1);
    MODIFY_FIELD(GPIOA->AFR[0], GPIO_AFR_L_AFSEL3, 1);

    // 9600 baud rate
    USART2->BRR = F_USART_CLOCK/9600;
    // No parity
    MODIFY_FIELD(USART2->CR1, USART_CR1_PCE, 0);
    // 8 data bits
    MODIFY_FIELD(USART2->CR1, USART_CR1_M, 0);
    // 1 Stop bit
    MODIFY_FIELD(USART2->CR2, USART_CR2_STOP, 0);

    // Enable transmitter and receiver and USART
    MODIFY_FIELD(USART2->CR1, USART_CR1_TE, 1);
    MODIFY_FIELD(USART2->CR1, USART_CR1_RE, 1);
    MODIFY_FIELD(USART2->CR1, USART_CR1_UE, 1);
}
```

Listing 8.8 Code to initialize USART2.

The initialization code in Listing 8.8 will configure USART2 to communicate with the ST-Link debug MCU. It does the following:

- Enables clock gating for USART2 and Port A.
- Connects PA3 and PA2 to USART2_RX and USART2_TX respectively.
- Sets the serial baud rate to 9600.
- Configures for eight data bits, no parity and one stop bit.
- Enables the transmitter, receiver and peripheral.

The USART is configured to be used as an UART device (without clock output pin). To simplify the configuration the program uses the UART initialization functions. Program access to the USART can be based on polling or interrupts. Polled communication is simple but does not share the CPU's time as interrupts do.

Polled Communication

To demonstrate polled communication, we will use the program in Listing 8.9. The program first initializes the USART with the function from Listing 8.8, sends out one exclamation point (!) as a greeting and then enters a loop. Within the loop, the program waits to receive a character. The function `UART_Receive` will return the character after it has been received by the USART. The character is incremented by one and sent back with the function `UART_Transmit`. We can try this program out on a PC with a terminal emulator program that communicates with the virtual serial port.

The function `UART_Transmit` in Listing 8.9 needs to ensure the transmit buffer is empty before trying to write a character to that buffer. This is indicated by a TXE flag value of one. At this point the code can write the data to transmit to `USARTx_TDR`. The code will not advance past the polling loop until the transmit buffer is empty.

The function `UART_Receive` in Listing 8.9 needs to ensure there is data in the receive buffer before trying to read it out. This is indicated by an RXNE value of one. At this point the code can read the received data from `USARTx_RDR`. The code will not advance past the polling loop until the receive buffer is not empty (i.e. a character is received).

```
void UART_Transmit(uint8_t data) {
    while ((USART2->ISR & USART_ISR_TXE) == 0)
        ;
    USART2->TDR = data;
}
uint8_t UART_Receive(void) {
    while ((USART2->ISR & USART_ISR_RXNE) == 0)
        ;
    return USART2->RDR;
}
int main(void) {
    uint8_t receive_data, transmit_data;

    Set_Clocks_To_48MHz();
    Init_USART2();
    UART_Transmit('!');
```

```

while (1) {
    receive_data = UART_Receive();
    transmit_data = receive_data + 1;
    UART_Transmit(transmit_data);
}

```

Listing 8.9 Code for polled serial communication to echo back the character with the code after the one received.

Figure 8.23 shows the logic analyzer demonstrating the program operating.

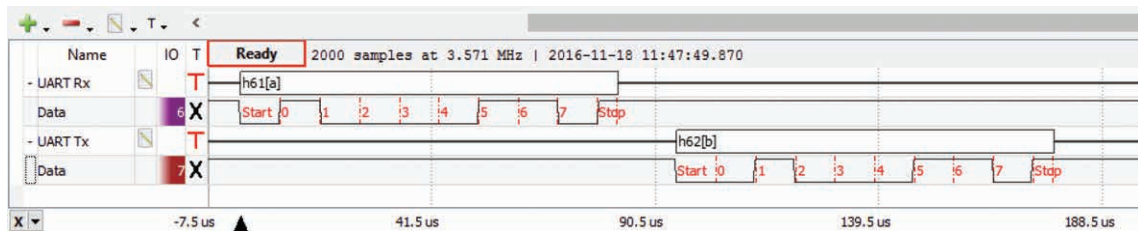


Figure 8.23 USART receives character “a,” program advances from “a” to next character code “b” by adding one and transmits it out.

Interrupt-Driven Communication

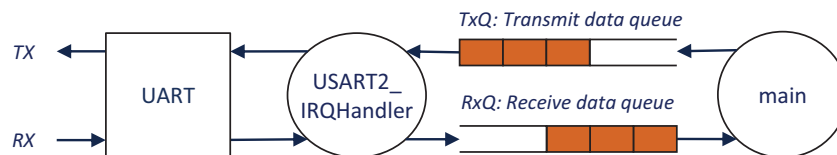


Figure 8.24 Overview of interrupt-driven USART communication.

Now we will use interrupt-driven communication to create a program that will respond to each character received with a message “You pressed x.” The program structure is shown in Figure 8.24. The receive data queue **RxQ** will store received data from the USART until the main thread is able to process it. The transmit data queue **TxQ** will store data to be transmitted until the USART is able to send it.

Figure 8.25 shows the software architecture in more detail. The two queues are accessed by the `USART2_IRQHandler` and two new functions: `USART_Q_Transmit_NonBlocking` and `USART_Q_Receive_Blocking`. We will use the USART interrupt so that the transmission is non-blocking and a new character can be received even if the transmission is not finished.

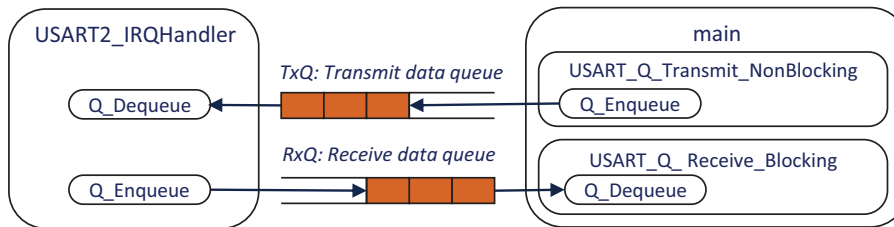


Figure 8.25 Detailed view of software used to access queues, showing function calls which access queues.

```
int main(void) {
    char buffer[80];
    uint8_t c;

    Set_Clocks_To_48MHz();
    Init_USART2(9600);
    Q_Init(&TxQ);
    Q_Init(&RxQ);
    strcpy(buffer, "Hello, world!\n\r");
    USART_Q_Transmit_NonBlocking(&TxQ, buffer, strlen(buffer));
    while (1) {
        USART_Q_Receive_Blocking(&RxQ, buffer, 1); // Request one character
        c = (uint8_t) buffer[0]; // Copy that character from buffer
        sprintf(buffer, "You pressed %c\n\r", c); // Prepare response
        USART_Q_Transmit_NonBlocking(&TxQ, buffer, strlen(buffer));
    }
}
```

Listing 8.10 Code in main thread for echoing serial input with interrupts.

The `main` code is shown in Listing 8.10. The code creates a message in the array called `buffer`. The library functions `strcpy` and `strlen` copy the string and determine its length. The `main` function then sends the message out the serial port by enqueueing it into `TxQ` with the function `USART_Q_Transmit_NonBlocking`, which is shown in Listing 8.11. This function is non-blocking because it returns after enqueueing the data to transmit, instead of blocking until the last byte of the message is sent.⁵ The function also sets the `TXEIE` field to one to ensure that an interrupt is generated when the transmit buffer is empty. As described below, the ISR disables this interrupt when there is no more data to transmit.

⁵ Note that if there is not enough space in `TxQ` to hold all the data, this function blocks until enough space opens up in the queue. Different applications might need to handle this situation differently.

```

void USART_Q_Transmit_NonBlocking(Q_T * Q, char * buf, int num_chars){
    while (num_chars > 0) {
        while (Q_Full(Q))
            ; // Temporary error: wait until queue is not full
        Q_Enqueue(Q, *buf);
        buf++;
        num_chars--;
    }
    // Ensure we'll get a transmit buffer empty interrupt
    MODIFY_FIELD(USART2->CR1, USART_CR1_TXEIE, 1);
}

```

Listing 8.11 `USART_Q_Transmit_NonBlocking` lets application load up transmit queue with `num_chars` of data for USART to send and then ensures transmission will occur. Non-blocking function returns after loading the last character into the queue, not later when the last character has been transmitted.

The `main` function then enters an infinite loop which calls `USART_Q_Receive_Blocking` (see Listing 8.12) to request one byte of data. This function copies all the bytes requested from `RxQ` into a buffer `buf`. If there are not enough characters in `RxQ`, this function blocks (waits) until enough have arrived, and then returns them. The received character is used to form a message to transmit and is loaded into `TxQ` for transmission using `USART_Q_Transmit_NonBlocking`.

```

void USART_Q_Receive_Blocking(Q_T * Q, char * buf, int num_chars){
    // This function gets num_chars, blocking if they aren't all ready yet
    while (num_chars > 0) {
        while (Q_Empty(&RxQ)) // Block until some data is received
            ;
        *buf = Q_Dequeue(&RxQ);
        num_chars--; // Got another character
        buf++; // Advance buffer pointer to space for next character
    }
}

```

Listing 8.12 `USART_Q_Receive_Blocking` lets application request multiple characters from receive queue, blocking (not returning) until all requested characters have been received.

Note that this example code explicitly blocks on the receive queue until a character is received. A practical approach would use a state machine or scheduler to share the processor rather than block, or return an error code indicating that there aren't enough characters available yet.

Listing 8.13 shows the ISR for `USART2`. The ISR first checks if a character has been received (`RXNE` is one). If so, it reads the character from the USART receive data register and enqueues it in `RxQ` if space is available (discarding it otherwise). The ISR then checks to see if the transmit buffer is empty (`TXE` is one). If so, it checks to see if there is more data in `TxQ` to send. If there is, it dequeues one data byte from `TxQ` and loads it in the USART transmit data register. If there is no more data to send, it disables the transmit buffer empty interrupt. The transmit buffer is still empty (`TXE=1`), but since `TXEIE` is cleared to zero it does not generate an interrupt.

```

void USART2_IRQHandler(void) {
    uint8_t ch;
    if (USART2->ISR & USART_ISR_RXNE) { // Receive buffer not empty
        // Reading from RDR will clear RXNE
        ch = USART2->RDR;
        if (!Q_Full(&RxQ)) {
            Q_Enqueue(&RxQ, ch);
        }
    }
    if (USART2->ISR & USART_ISR_TXE) { // Transmit buffer empty
        // Writing to TDR will clear TXE
        if (!Q_Empty(&TxQ)) { // More data to send
            USART2->TDR = Q_Dequeue(&TxQ);
        } else { // Disable transmitter interrupt
            MODIFY_FIELD(USART2->CR1, USART_CR1_TXEIE, 0);
        }
    }
}
}

```

Listing 8.13 Interrupt handler for USART2 transfers data between RDR and RxQ or TDR and TxQ.

```

#define F_USART_CLOCK (48UL*1000UL*1000UL) // 48 MHz
Q_T RxQ, TxQ;

void Init_USART2(int baud_rate) {
    RCC->APB1ENR |= RCC_APB1ENR_USART2EN;
    RCC->AHBENR |= RCC_AHBENR_GPIOAEN;

    // GPIO A pin 2 and 3 in alternate function 1 (USART2)
    // Set mode field to 2 for alternate function
    MODIFY_FIELD(GPIOA->MODER, GPIO_MODER_MODER2, ESF_GPIO_MODER_ALT_FUNC);
    MODIFY_FIELD(GPIOA->MODER, GPIO_MODER_MODER3, ESF_GPIO_MODER_ALT_FUNC);
    // Select USART2 (AF = 1) as alternate function
    MODIFY_FIELD(GPIOA->AFR[0], GPIO_AFRL_AFSEL2, 1);
    MODIFY_FIELD(GPIOA->AFR[0], GPIO_AFRL_AFSEL3, 1);

    USART2->BRR = F_USART_CLOCK/baud_rate;
    MODIFY_FIELD(USART2->CR1, USART_CR1_PCE, 0); // No parity
    MODIFY_FIELD(USART2->CR1, USART_CR1_M, 0); // 8 data bits
    MODIFY_FIELD(USART2->CR2, USART_CR2_STOP, 0); // 1 Stop bit

    // Enable interrupt generation
    MODIFY_FIELD(USART2->CR1, USART_CR1_TXEIE, 1);
    MODIFY_FIELD(USART2->CR1, USART_CR1_RXNEIE, 1);

    // Enable transmitter and receiver and USART
    MODIFY_FIELD(USART2->CR1, USART_CR1_TE, 1);
    MODIFY_FIELD(USART2->CR1, USART_CR1_RE, 1);
    MODIFY_FIELD(USART2->CR1, USART_CR1_UE, 1);

    // Enable USART2 interrupts in NVIC
    NVIC_SetPriority(USART2_IRQn, 2);
    NVIC_ClearPendingIRQ(USART2_IRQn);
    NVIC_EnableIRQ(USART2_IRQn);
}

```

Listing 8.14 Updated version of Init_USART2() enables interrupts and takes baud rate as a parameter.

